

align(center + horizon)[block(width: 15cm, inset: (y: 1.1cm), stroke: (top: 2.5pt + rgb(26, 54, 93),
bottom: 0.75pt + rgb(203, 213, 225)),) [align(center)[text(size: 10.5pt, tracking: 0.35em, fill:
rgb(71, 85, 105))[大 作 业] v(0.55cm) text(size: 26pt, weight: “bold”, fill: rgb(15, 23, 42))[揭棋对弈
程序设计] v(0.65cm) text(size: 19pt, weight: “medium”, fill: rgb(30, 64, 175))[AI 算法设计]
v(0.35cm) if subtitle != “” [text(size: 13pt, fill: rgb(100, 116, 139))[Artificial Intelligence Design –
Alpha-Beta、Belief Sampling、评估函数] v(0.35cm)]]]

v(1.5cm)

box(width: 13cm, inset: (x: 1.2cm, y: 0.95cm), fill: rgb(248, 250, 252), radius: 6pt, stroke: 0.75pt +
rgb(226, 232, 240),) [align(center)[text(size: 11pt, weight: “bold”, fill: rgb(51, 65, 85))[小组成员]
v(0.55cm) grid(columns: (1fr, 1fr), column-gutter: 1.6cm, row-gutter: 0.65cm, align: center +
horizon, [text(size: 13pt, weight: “bold”)[张恒基 (组长)]
text(size: 12pt, fill: rgb(100, 116, 139))[2024211301 / 2024210926]], [text(size: 13pt, weight: “bold”)
[秦博宇]
text(size: 12pt, fill: rgb(100, 116, 139))[2024211302 / 2024210940]], [text(size: 13pt, weight: “bold”)
[陈艺博]
text(size: 12pt, fill: rgb(100, 116, 139))[2024211302 / 2024210931]], [text(size: 13pt, weight: “bold”)
[陈雨飞]
text(size: 12pt, fill: rgb(100, 116, 139))[2024211301 / 2024210918]],)]]

v(1cm) text(size: 12pt, fill: rgb(100, 116, 139))[技术设计 · 2026-06-18] v(0.5cm) text(size: 10.5pt,
fill: rgb(148, 163, 184))[北京邮电大学 · 计算机学院]]

设计目标与约束

约束	实现
走法必须合法	通过 generateLegalMoves + 最终 fallback 保证
不能透视对手暗子	仅用 createAiPublicView, 对手暗子 type=UNKNOWN
必须按时返回	timeLimitMs 超时后取已有最佳走法 fallback
三档难度可感知	Easy < 500ms; Medium 5s 搜索; Hard 多采样 5s

三档 AI 对照

维度	Easy	Medium	Hard
实现类	EasyRuleBot	AlphaBetaBot → JieqiAgent	BeliefAlphaBetaBot
搜索算法	无（启发式选择）	Alpha-Beta + 置换表	Belief Sampling + Alpha-Beta
搜索深度	0	迭代加深到深度 ≥ 8	每次搜索深度 4-6
暗子处理	公开视角	公开视角	对对手暗子采样确定化
时间预算	< 500ms	5s	5s（24 次搜索）
随机性	30% 全随机 + 70% TopK	无（纯搜索）	采样随机性

搜索算法详解

Alpha-Beta 剪枝

核心搜索引擎 OptimizedAlphaBeta 实现标准 negamax 框架：

```
function alphaBeta(board, depth, alpha, beta, color, evalBias):
    if time budget exhausted: return evaluate(board, color)
    if depth == 0: return quiescenceSearch(board, color, alpha, beta, evalBias)

    probe TT; if usable: return ttScore

    for each move in orderMoves(moves):
        board.makeMove(move); undo snapshot
        score = -alphaBeta(board, depth-1, -beta, -alpha, -color, evalBias)
        board.undoMove(snapshot)

        if score >= beta: store killer/history; return beta // cut-off
        if score > alpha: alpha = score; update PV

    store TT; return alpha
```

迭代加深

深度 1→20 逐层加深，低深度结果用于 move ordering：

- 浅层搜索成本低，结果指导深层搜索的走法排序
- 任何一层超时都可返回已有最佳走法
- 深层搜索复用浅层的 PV 走法

置换表（Transposition Table）

- ZobristHash 计算 64-bit 局面键

- TT 字典: $\text{key} \rightarrow (\text{depth}, \text{flag}, \text{score}, \text{bestMove})$
- flag: EXACT / ALPHA ($\leq \alpha$) / BETA ($\geq \beta$)
- TT 命中且 $\text{depth} \geq \text{current depth}$ 时直接返回

杀手启发 (Killer Heuristic)

- 记录每个深度造成 beta cut-off 的走法 (2 个槽位)
- 相同深度的兄弟节点优先尝试杀手走法
- 实现: KillerHeuristic.java

历史启发 (History Heuristic)

- 推高 alpha 的走法更新历史表分值
- 历史高分走法在 move ordering 中优先
- 实现: HistoryHeuristic.java

Aspiration Window

以上次搜索分数 center 为中心, 缩窄 alpha-beta 窗口:

```
ASPIRATION_WINDOW = 80
for depth > 1:
    alpha = prevScore - ASPIRATION_WINDOW
    beta = prevScore + ASPIRATION_WINDOW
    score = searchAtDepth(depth, alpha, beta)
    if fail-low: re-search with alpha = -INF
    if fail-high: re-search with beta = +INF
```

静态搜索 (Quiescence Search)

仅在吃子中扩展搜索 (delta 剪枝), 最多 3 层:

```
function quiescenceSearch(board, color, alpha, beta, evalBias):
    standPat = evaluate(board, color) + evalBias
    if standPat >= beta: return beta
    if standPat > alpha: alpha = standPat

    for each capture in orderCaptures(moves):
        if SEE(move) < 0: continue // losing capture
        board.makeMove(move)
        score = -quiescenceSearch(board, -color, -beta, -alpha, evalBias)
        board.undoMove(move)
        // ... alpha-beta update
```

SEE 静态交换评估

模拟目标格连续交换, 净得分 ≥ 0 才放入静态搜索。

长将规避

repetitionRisk 惩罚: 长将局面得分被压低, AI 主动规避重复将军。

Late Move Reductions (LMR)

- 条件: $\text{quiet move} + \text{searched} \geq 4 + \text{depth} \geq 3 + \text{!isPV} \rightarrow \text{reduction}=1$
- 减深搜索若推高 alpha, 用完整深度 re-search

评估函数详解

EnhancedEvaluator 综合 7 个维度:

评估维度

维度	权重	说明
子力 (Material)	核心	明子固定值 + 暗子期望值；将 4000，车 800，马炮 350，兵/卒 80-120
位置 (Position)	增量	车马炮兵的位置分表 (PST)，中心/过河加分
机动性 (Mobility)	辅助	合法走法数量 × 权重因子
将帅安全 (King Safety)	中局重要	士象护卫 + 宫心控制
威胁 (Threats)	战术	MVV-LVA 威胁评估
兵形 (Pawn Structure)	辅助	兵卒相邻加成，过河奖励
残局猎杀 (King Hunt)	残局权重	子力减少时压迫对方将位得分

```
set text(size: 11pt, fill: rgb(3, 105, 161)) body
```

揭棋隐藏信息处理

AI 公开视角

- board.createAiPublicView(color):
- 己方暗子保留真实 type (AI 知道自己手下有什么兵)
 - 对手暗子 type 置 UNKNOWN
 - 明子和将帅双方可见

```
set text(size: 11pt, fill: rgb(3, 105, 161)) body
```

Belief Sampling (Hard 档核心)

步骤	操作
1. 构建公开视角	从 AI 视角复制棋盘
2. 确定子力池	从已揭示对手棋子反推剩余暗子 pool
3. 随机分配	对每个对手暗子，从 pool 随机抽取 type 写入采样 Board
4. Alpha-Beta 搜索	对确定化采样局面上标准 AB 搜索
5. 期望收益	多采样后，选择期望收益最大的候选走法

```
for each candidate move (TopK by quick-eval):
    expectedScore = 0
    for each sample s in 1..N:
        sampleBoard = BoardSampler.fromPublicView(publicBoard)
        score = alphaBeta(sampleBoard, ...)
        expectedScore += score
    select move with max(expectedScore)
```

- 采样数根据时间预算动态调整：
- budget >= 5000ms: 24 采样
 - budget < 3000ms: 降级到 2 采样 + 4 候选

与标准 Alpha-Beta 的关系

- 每个采样局面上做标准 Alpha-Beta（确定化后就是完全信息）
- 不同采样的搜索结果不能跨采样复用（置换表每个采样重建）
- 期望收益最大的走法是 Hard AI 的选择

Agent 编排

AgentOrchestrator 按优先级调度子 Agent：

Agent	职责	触发条件
ProbabilityAgent	提供暗子概率偏差 evalBias	始终执行
EndgameAgent	残局精确搜索（深度优先）	双方子力 <= 阈值
SearchAgent	主搜索（Alpha-Beta + 迭代加深）	默认

evalBias 从 ProbabilityAgent 流经 SearchAgent → OptimizedAlphaBeta.search() → quiescenceSearch standPat。

已知限制

限制	说明
Hard 搜索浅	Belief Sampling 在预算紧张时每个采样深度受限（4-6）
TT 缓存污染	采样场景下置换表可能有噪声；每个采样独立 new OptimizedAlphaBeta()
残局库缺失	无预计算残局数据库，依赖搜索到底
概率模型简化	暗子期望值假设均匀分布，未建模对手意图